

# visualDraw

## Gestural AI-Powered Canvas — Technical Documentation

Version 1.0 · May 25, 2026 · Confidential

This document provides comprehensive technical documentation for **visualDraw**, a browser-based gestural drawing application powered by real-time hand tracking. It is intended for developers, evaluators, and academic reviewers.

# Table of Contents

---

- 1. Project Overview
- 2. System Architecture
  - 2.1 Technology Stack
  - 2.2 Project Structure
- 3. Core Modules
  - 3.1 Drawing Engine
  - 3.2 Gesture Recognition System
  - 3.3 Export Engine
- 4. Hand Gesture Reference
- 5. User Interface Controls
- 6. Browser Compatibility
- 7. Installation & Setup
- 8. API & CDN Dependencies
- 9. Future Roadmap
- 10. License & Author

# 1. Project Overview

**visualDraw** is a browser-native, AI-assisted gestural canvas application that enables users to create pixel-grid artwork using hand gestures captured via a standard webcam. The system leverages **MediaPipe Hands** for sub-20 ms hand-landmark inference and renders results on an **HTML5 Canvas** element, delivering a low-latency, touchless drawing experience.

The application is designed to be lightweight, requiring no server-side infrastructure. All processing is performed client-side in the browser, making it immediately deployable via any static web host or local development server.

**Key Value Proposition:** visualDraw eliminates the need for physical input devices by translating natural hand gestures into precise digital strokes, opening creative computing to accessibility-focused and gesture-first interaction paradigms.

## 1.1 Feature Summary

| Feature                 | Description                                          |
|-------------------------|------------------------------------------------------|
| Real-Time Hand Tracking | MediaPipe Hands pipeline running at up to 60 FPS     |
| Gesture Drawing         | Right-hand pinch gesture triggers pixel drawing      |
| Gesture Erasing         | Left-hand open-palm gesture activates eraser         |
| Dynamic Grid            | Configurable grid from 5×5 up to 50×50 cells         |
| Brush Shapes            | Square, Circle, and Diamond brush profiles           |
| Adjustable Parameters   | Brush size, opacity, pinch sensitivity, eraser size  |
| Export Formats          | PNG, JPEG, SVG, and JSON matrix export               |
| Multi-Camera Support    | Device enumeration with live camera switching        |
| Canvas Utilities        | Flip, Invert, Fill, and coordinate overlay tools     |
| Live FPS Monitor        | Real-time frame-rate display for performance insight |

## 2. System Architecture

visualDraw follows a modular, event-driven architecture entirely contained within a single HTML document. The system is composed of three primary subsystems — the **Gesture Recognition System**, the **Drawing Engine**, and the **Export Engine** — each communicating through a shared application state object.

### 2.1 Technology Stack

| Layer      | Technology                | Purpose                                       |
|------------|---------------------------|-----------------------------------------------|
| Markup     | HTML5                     | Structural document and canvas host           |
| Styling    | CSS3 + TailwindCSS        | Futuristic UI design and responsive layout    |
| Logic      | Vanilla JavaScript (ES6+) | Application state, rendering, gesture routing |
| CV / ML    | MediaPipe Hands           | Real-time 21-point hand landmark detection    |
| Rendering  | HTML5 Canvas API          | 2-D pixel-grid drawing surface                |
| Typography | Google Fonts              | UI font assets (CDN-delivered)                |

### 2.2 Project Structure

The repository follows a minimal flat structure suitable for static hosting:

```
visualDraw/  
  index.html      # Application entry point (self-contained)  
  README.md       # Project overview and setup guide  
  assets/         # Static assets (screenshots, icons)
```

All JavaScript, CSS, and HTML reside in **index.html**. External dependencies are loaded from CDN at runtime; no build step or package manager is required.

## 3. Core Modules

### 3.1 Drawing Engine

The Drawing Engine manages a logical pixel grid that maps screen coordinates to discrete cell positions. On each animation frame, the engine evaluates the current gesture state, determines which cells fall within the active brush footprint, and composites them onto the HTML5 Canvas with the configured opacity.

- Grid System:** The canvas is subdivided into an N×N logical grid ( $N \in [5, 50]$ ). Each cell stores an RGBA colour value.
- Brush Shapes:** Three brush profiles are supported — *Square* (axis-aligned bounding box fill), *Circle* (Euclidean distance mask), and *Diamond* (L1 distance mask).
- Opacity:** Per-stroke alpha blending is applied, enabling layered shading effects.
- Real-Time Rendering:** The canvas is redrawn on every MediaPipe result callback, maintaining synchronisation with the camera feed.

### 3.2 Gesture Recognition System

Gesture recognition is built on top of the MediaPipe Hands solution, which provides 21 3-D landmarks per detected hand at up to 60 frames per second. The application subscribes to the onResults callback and classifies each hand independently.

| Gesture   | Hand  | Detection Method                                                           | Action |
|-----------|-------|----------------------------------------------------------------------------|--------|
| Pinch     | Right | Distance between thumb tip (L4) and index tip (L8) < sensitivity threshold | Draw   |
| Open Palm | Left  | All five finger tips extended; mean tip-to-wrist distance above threshold  | Erase  |

**Note:** Pinch sensitivity is user-adjustable via the Sensitivity slider. Lowering sensitivity requires a tighter pinch to trigger drawing, reducing accidental activation.

### 3.3 Export Engine

The Export Engine serialises the current canvas state into multiple formats for downstream use in design tools, data pipelines, or web publishing.

| Format | Method                                | Use Case                                              |
|--------|---------------------------------------|-------------------------------------------------------|
| PNG    | Canvas.toDataURL('image/png')         | Lossless raster; best for sharing and printing        |
| JPEG   | Canvas.toDataURL('image/jpeg', 0.92)  | Compressed raster; smaller file size                  |
| SVG    | Programmatic rect generation per cell | Scalable vector; suitable for design tools            |
| JSON   | Grid matrix serialisation             | Raw data export for AI training or programmatic reuse |

## 4. Hand Gesture Reference

The table below defines the complete gesture vocabulary supported in version 1.0. Future releases may extend this vocabulary through the Gesture Customisation module described in Section 9.

| Gesture ID | Hand  | Visual Cue                                | Mapped Action | Notes                  |
|------------|-------|-------------------------------------------|---------------|------------------------|
| G-01       | Right | Thumb & index fingertips brought together | Draw          | Sensitivity adjustable |
| G-02       | Left  | All fingers extended, palm facing camera  | Erase         | Eraser size adjustable |

**Tip:** For best recognition accuracy, ensure even lighting on the hand, keep the hand within 40–80 cm of the camera, and avoid complex backgrounds. Chromium-based browsers are recommended for optimal MediaPipe performance.

## 5. User Interface Controls

The control panel provides the following interactive elements. All controls update application state in real time without requiring a page reload.

| Control            | Type         | Range / Options           | Function                                    |
|--------------------|--------------|---------------------------|---------------------------------------------|
| Grid Size          | Slider       | 5 – 50 (integer)          | Sets the N×N resolution of the drawing grid |
| Brush Size         | Slider       | 1 – 10 (integer)          | Controls the brush footprint in grid cells  |
| Opacity            | Slider       | 0.1 – 1.0 (float)         | Sets per-stroke alpha value                 |
| Pinch Sensitivity  | Slider       | Application-defined       | Threshold distance for pinch detection      |
| Eraser Size        | Slider       | 1 – 10 (integer)          | Controls eraser footprint in grid cells     |
| Brush Shape        | Button Group | Square / Circle / Diamond | Selects active brush profile                |
| Export (PNG)       | Button       | —                         | Downloads canvas as PNG image               |
| Export (JPEG)      | Button       | —                         | Downloads canvas as JPEG image              |
| Export (SVG)       | Button       | —                         | Downloads canvas as SVG vector file         |
| Export (JSON)      | Button       | —                         | Downloads grid matrix as JSON               |
| Flip Canvas        | Button       | —                         | Mirrors canvas horizontally                 |
| Invert Colours     | Button       | —                         | Inverts all cell colours                    |
| Fill Canvas        | Toggle       | On / Off                  | Fills all cells with current colour         |
| Grid Overlay       | Toggle       | On / Off                  | Shows or hides grid lines                   |
| Coordinate Overlay | Toggle       | On / Off                  | Displays cell (x, y) coordinates            |
| Camera Select      | Dropdown     | Detected devices          | Switches active camera device               |

## 6. Browser Compatibility

visualDraw relies on **getUserMedia** (WebRTC), **WebAssembly** (required by MediaPipe), and the **Canvas 2D API**. The following browsers have been validated:

| Browser             | Engine     | Support Level | Notes                                                    |
|---------------------|------------|---------------|----------------------------------------------------------|
| Google Chrome 110+  | Blink / V8 | Full          | Recommended; best MediaPipe WASM performance             |
| Microsoft Edge 110+ | Blink / V8 | Full          | Chromium-based; equivalent to Chrome                     |
| Brave 1.50+         | Blink / V8 | Full          | Disable aggressive fingerprint shields if camera blocked |
| Mozilla Firefox     | Gecko      | Partial       | WebAssembly SIMD support varies; not recommended         |
| Apple Safari        | WebKit     | Not tested    | MediaPipe WASM support limited; use Chrome on macOS      |



## 7. Installation & Setup

---

visualDraw requires no build toolchain. Follow the steps below to run the application locally.

### Step 1 — Clone the Repository

Clone the project from the version control repository:

```
git clone https://github.com/yourusername/visualDraw.git
cd visualDraw
```

### Step 2 — Serve Locally

Because the application uses the getUserMedia API, it must be served over HTTP(S) — opening index.html directly as a file:// URL will cause the camera to be blocked by the browser. Use any of the following methods:

```
# Option A: VS Code Live Server extension
# Right-click index.html → 'Open with Live Server'

# Option B: Python built-in server
python3 -m http.server 8080

# Option C: Node.js http-server
npm http-server -p 8080
```

### Step 3 — Grant Camera Permission

Navigate to <http://localhost:8080> in a Chromium-based browser. When prompted, allow camera access. The hand-tracking pipeline will initialise automatically.

## 8. API & CDN Dependencies

All external libraries are loaded from public CDNs. No npm or pip installation is required. The following scripts must be included in <head> before the application script:



| Library                  | CDN Provider         | Purpose                                    |
|--------------------------|----------------------|--------------------------------------------|
| @mediapipe/camera_utils  | jsDelivr             | Webcam stream management for MediaPipe     |
| @mediapipe/drawing_utils | jsDelivr             | Landmark rendering utilities               |
| @mediapipe/hands         | jsDelivr             | Core hand-landmark ML model and inference  |
| TailwindCSS              | cdn.tailwindcss.com  | Utility-first CSS framework for UI styling |
| Google Fonts             | fonts.googleapis.com | Web font delivery                          |

## 9. Future Roadmap

The following enhancements are planned for subsequent releases. Items are listed in approximate priority order based on user impact and implementation complexity.

| Priority | Feature               | Description                                                             |
|----------|-----------------------|-------------------------------------------------------------------------|
| High     | Undo / Redo Stack     | Command-pattern history buffer for reversible drawing operations        |
| High     | Save / Load Projects  | Persist and restore canvas state via localStorage or file download      |
| Medium   | AI Shape Recognition  | CNN-based post-processing to snap free-hand strokes to canonical shapes |
| Medium   | Gesture Customisation | User-configurable gesture-to-action bindings                            |
| Medium   | Mobile Optimisation   | Touch-event fallback and responsive layout for tablet devices           |
| Low      | Multiplayer Canvas    | WebSocket-based collaborative drawing with conflict resolution          |

## 10. License & Author

### 10.1 License

visualDraw is released under the **MIT License**. Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files, to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, subject to the following conditions: the above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED 'AS IS', WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NON-INFRINGEMENT.

### 10.2 Author

|           |                                                                                           |
|-----------|-------------------------------------------------------------------------------------------|
| Name      | Ashwin Jain                                                                               |
| Role      | Web Developer & Creative Programmer                                                       |
| Portfolio | <a href="http://codemaster.infinityfreeapp.com">http://codemaster.infinityfreeapp.com</a> |

### 10.3 Acknowledgements

- **MediaPipe** — Google's open-source ML framework for real-time perception pipelines.
- **TailwindCSS** — Utility-first CSS framework by Tailwind Labs.
- **Google Fonts** — Free, open-source font hosting infrastructure.
- **HTML5 Canvas API** — W3C standard 2-D graphics API for web browsers.